★ GitOps Delivery for shop-api

Level 17 capstone ~10 hours

1 Goal and outcome

Build a complete GitOps delivery system on a laptop. By the end you will have:

- a local `kind` cluster running **Argo CD**;
- a **GitOps repository** (`shop-gitops`) with a Kustomize base and `dev` / `staging` / `prod` overlays;
- an **App of Apps** root and an **ApplicationSet** that generate one Argo CD Application per environment, with **automated sync, prune and self-heal**;
- a **PreSync / PostSync hook** pair and sync waves;
- **promotion dev -> staging -> prod by pull request**, gated by CODEOWNERS for prod;
- a **CI workflow** that builds an image and bumps the dev tag in the GitOps repo;
- a **sealed secret** (or External Secrets) instead of plain-text credentials;
- a rehearsed **rollback with** `git revert` and an emergency procedure.

This exercises every topic: Desired State Management §17.1, Argo CD and Flux §17.2, Continuous Reconciliation §17.3, Environment Promotion §17.4, Application Synchronization §17.5, Rollback Through Git §17.6 and The Full Flow: CI to GitOps §17.7.

2 Architecture

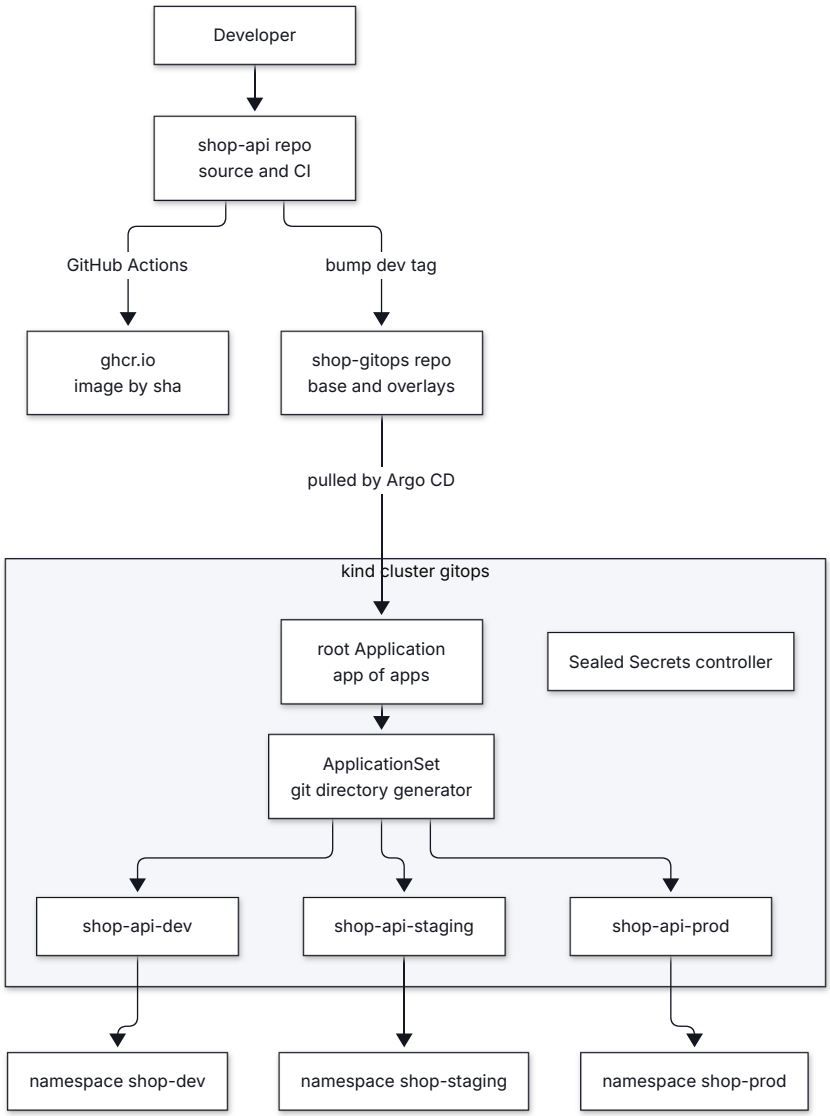


Figure 17-1

Promotion and rollback path:

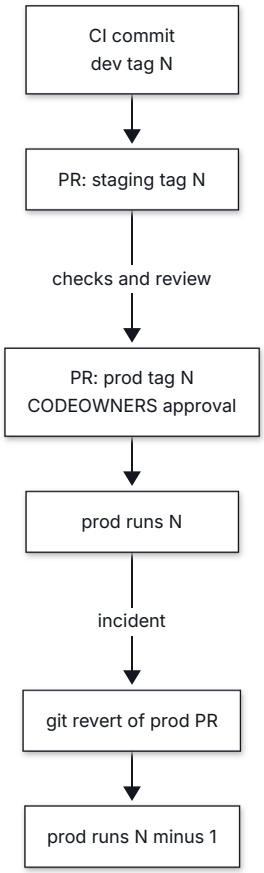


Figure 17-2

3 Prerequisites and cost

- Docker, `kind`, `kubectl`, `git`, `helm`, `kustomize` (standalone), `gh` (GitHub CLI), optional `argocd` CLI and `kubeseal`.
- A GitHub account, two empty repositories you create: `shop-gitops` (public is simplest) and, optionally, `shop-api` for the CI part.
- About 4 GB of free RAM and 10 GB of disk for the kind node and images.
- **Cost:** free. GitHub Actions free-tier minutes are enough. No cloud resources are created; teardown is `kind delete cluster`.
- To keep the sandbox simple the capstone deploys the public `nginx` image as a stand-in for `shop-api`. If you built `shop-api` in earlier levels, substitute your own image.

- Outputs shown below are illustrative; yours will differ in names, ages and timings.

4 Build it step by step

Milestone 1 - Cluster and Argo CD (topics 1-2)

```bash
kind create cluster --name gitops
kubectl create namespace argocd
kubectl apply -n argocd --server-side --force-conflicts \
  -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl -n argocd rollout status deploy/argocd-server --timeout=300s
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath='{.data.password}' |
  base64 -d; echo
kubectl -n argocd port-forward svc/argocd-server 8080:443 &
```

Browse to `https://localhost:8080`, log in as `admin`. (For anything beyond a sandbox, pin a specific Argo CD release instead of `stable`.)

Milestone 2 - The GitOps repo (topic 1)

```bash
git clone https://github.com/<YOUR_USER>/shop-gitops.git && cd shop-gitops
mkdir -p apps/shop-api/{base,overlays/{dev,staging,prod}} clusters/local/apps .github
```

Base manifests:

```yaml
# apps/shop-api/base/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shop-api
  labels:
    app: shop-api
spec:
  replicas: 1
  selector:
    matchLabels:
      app: shop-api
  template:
    metadata:
      labels:
        app: shop-api
    spec:
      containers:
        - name: shop-api
```

```
          image: nginx:1.26
          ports:
            - containerPort: 80
          readinessProbe:
            httpGet:
              path: /
              port: 80
```

YAML

```yaml
# apps/shop-api/base/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: shop-api
spec:
  selector:
    app: shop-api
  ports:
    - port: 80
      targetPort: 80
```

YAML

```yaml
# apps/shop-api/base/presync-job.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate
  annotations:
    argocd.argoproj.io/hook: PreSync
    argocd.argoproj.io/hook-delete-policy: BeforeHookCreation,HookSucceeded
spec:
  backoffLimit: 0
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: migrate
          image: busybox:1.36
          command: ["sh", "-c", "echo running migrations && sleep 3"]
```

YAML

```yaml
# apps/shop-api/base/postsync-job.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: smoke-test
  annotations:
    argocd.argoproj.io/hook: PostSync
```

```
    argocd.argoproj.io/hook-delete-policy: BeforeHookCreation,HookSucceeded
spec:
  backoffLimit: 0
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: smoke
          image: busybox:1.36
          command: ["sh", "-c", "wget -qO- http://shop-api && echo smoke test passed"]
```

YAML

```yaml
# apps/shop-api/base/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - deployment.yaml
  - service.yaml
  - presync-job.yaml
  - postsync-job.yaml
```

Overlays (shown for dev; staging uses namespace `shop-staging`, 2 replicas, tag `1.26`; prod uses `shop-prod`, 3 replicas, tag `1.26`):

YAML

```yaml
# apps/shop-api/overlays/dev/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: shop-dev
resources:
  - ../../base
images:
  - name: nginx
    newTag: "1.27"
```

YAML

```yaml
# apps/shop-api/overlays/staging/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: shop-staging
resources:
  - ../../base
replicas:
  - name: shop-api
    count: 2
images:
  - name: nginx
    newTag: "1.26"
```

YAML

```yaml
# apps/shop-api/overlays/prod/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: shop-prod
resources:
  - ../../base
replicas:
  - name: shop-api
    count: 3
images:
  - name: nginx
    newTag: "1.26"
```

Render and commit:

BASH

```bash
for e in dev staging prod; do kubectl kustomize apps/shop-api/overlays/$e | grep -E
  'namespace:|replicas:|image: nginx'; done
git add . && git commit -m "feat: shop-api base and overlays" && git push
```

Milestone 3 - App of Apps and ApplicationSet (topics 2, 3, 5)

YAML

```yaml
# clusters/local/root-app.yaml  (applied once by hand)
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: root
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/<YOUR_USER>/shop-gitops.git
```

```
    targetRevision: main
    path: clusters/local/apps
  destination:
    server: https://kubernetes.default.svc
    namespace: argocd
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

YAML

```yaml
# clusters/local/apps/shop-api-appset.yaml
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: shop-api-envs
  namespace: argocd
spec:
  goTemplate: true
  goTemplateOptions: ["missingkey=error"]
  generators:
    - git:
        repoURL: https://github.com/<YOUR_USER>/shop-gitops.git
        revision: main
        directories:
          - path: apps/shop-api/overlays/*
  template:
    metadata:
      name: 'shop-api-{{ .path.basename }}'
    spec:
      project: default
      source:
        repoURL: https://github.com/<YOUR_USER>/shop-gitops.git
        targetRevision: main
        path: '{{ .path.path }}'
      destination:
        server: https://kubernetes.default.svc
        namespace: 'shop-{{ .path.basename }}'
      syncPolicy:
        automated:
          prune: true
          selfHeal: true
        syncOptions:
          - CreateNamespace=true
```

BASH

```bash
git add . && git commit -m "feat: root app and applicationset" && git push
kubectl apply -f clusters/local/root-app.yaml
kubectl -n argocd get applications
```

Expect (illustrative) `root`, `shop-api-dev`, `shop-api-staging`, `shop-api-prod`, all `Synced` and `Healthy` after a couple of minutes.

Milestone 4 - Reconciliation behaviour (topic 3)

```bash
kubectl -n shop-dev scale deploy/shop-api --replicas=5   # drift
kubectl -n shop-dev get deploy shop-api -w               # returns to 1
```

Milestone 5 - Promotion by pull request (topic 4)

```bash
git switch -c promote-staging-1.27
(cd apps/shop-api/overlays/staging && kustomize edit set image nginx=nginx:1.27)
git commit -am "promote shop-api 1.27 to staging" && git push -u origin HEAD
gh pr create --fill
```

Add the prod gate:

```
# .github/CODEOWNERS
/apps/shop-api/overlays/prod/ @<YOUR_USER>
```

and a validation workflow:

```yaml
# .github/workflows/validate.yaml
name: validate
on:
  pull_request:
jobs:
  render:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Render every overlay
        run: |
          for d in apps/*/overlays/*; do
            echo "== $d"
            kubectl kustomize "$d" > /dev/null
          done
```

Milestone 6 - Secrets and CI hand-off (topic 7)

```bash
helm repo add sealed-secrets https://bitnami-labs.github.io/sealed-secrets && helm repo
  update
helm install sealed-secrets sealed-secrets/sealed-secrets -n kube-system
kubectl create secret generic shop-api-db -n shop-dev \
  --from-literal=DATABASE_URL='postgres://shop:CHANGE_ME@10.0.0.5:5432/shop' --dry-
  run=client -o yaml \
 | kubeseal --controller-name sealed-secrets --controller-namespace kube-system --format
  yaml \
 > apps/shop-api/overlays/dev/sealed-db.yaml
```

Add `sealed-db.yaml` to the dev overlay's `resources:`. For CI, add the workflow from The Full Flow §17.7 to your `shop-api` repo (token secret `GITOPS_TOKEN`).

Milestone 7 - Rollback (topic 6)

Promote a broken tag to prod by PR (for example `nginx:does-not-exist`), watch it degrade, then:

```bash
git switch main && git pull
git revert --no-edit HEAD
git push        # or via a PR if main is protected
```

Then rehearse the emergency path: `argocd app set shop-api-prod --sync-policy none`, `argocd app rollback ...`, fix Git, re-enable automation.

5 Tasks

- [ ] **T1 — Stand up the platform** *(≈60 min, beginner; Milestone 1; topic 2)*
  - **Goal:** Argo CD running on kind.
  - **Steps:**
    1. Run Milestone 1.
    2. Log in to the UI and change the admin password.
  - **Done when:** `kubectl -n argocd get pods` is all `Running` and the UI loads.
- [ ] **T2 — Create the repo and render the overlays** *(≈60 min, beginner; Milestone 2; topic 1)*
  - **Goal:** a base plus three overlays that differ only in namespace, replicas and tag.
  - **Steps:**
    1. Create all files; run the render loop.

2. Diff dev against prod rendered output.

- **Done when:** the diff shows only namespace, replica count and image tag differences, and the repo is pushed.

- [ ] **T3 — Bootstrap with App of Apps and ApplicationSet** *(≈60 min, intermediate; Milestone 3; topics 2 and 5)*
  - **Goal:** one `kubectl apply` creates every environment.
  - **Steps:**
    1. Apply `root-app.yaml`.
    2. Wait for the three generated Applications.
    3. Add `overlays/qa` and watch `shop-api-qa` appear; then remove it.
  - **Done when:** four Applications existed at one point and the cluster ended with three, all without hand-writing an Application.

- [ ] **T4 — Prove reconciliation** *(≈45 min, intermediate; Milestone 4; topic 3)*
  - **Goal:** demonstrate drift correction and prune.
  - **Steps:**
    1. Scale the dev Deployment by hand and watch it revert.
    2. Add a ConfigMap to the base, let it sync, then delete it from Git and watch it be pruned.
    3. Add `ignoreDifferences` for `/spec/replicas` to the ApplicationSet template, then show manual scaling is no longer reverted.
  - **Done when:** you have a screenshot or terminal log of all three behaviours.

- [ ] **T5 — Observe hooks and waves** *(≈45 min, intermediate; Milestone 2-3; topic 5)*
  - **Goal:** see ordered syncs.
  - **Steps:**
    1. Bump the dev tag and push.
    2. Watch `kubectl -n shop-dev get jobs,pods -w` and the sync operation panel.
    3. Make the PreSync command `exit 1` and observe the sync fail without replacing the old pods; fix it.
  - **Done when:** the failed PreSync blocks the rollout and the fixed one lets it through.

- [ ] **T6 — Promote dev -> staging -> prod by PR** *(≈90 min, advanced; Milestone 5; topic 4)*
  - **Goal:** a full gated promotion.
  - **Steps:**
    1. Promote `nginx:1.27` to staging by PR; the `validate` check must pass.

2. Promote it to prod; add CODEOWNERS and enable branch protection requiring code-owner review.
3. Verify each namespace runs the intended tag.

- **Done when:** `git log` for each overlay shows one promotion PR and `kubectl` shows 1.27 in all three namespaces.

☐ **T7 — Secrets without plain text** *(≈60 min, advanced; Milestone 6; topic 7)*

- **Goal:** deliver a credential via Git safely.
- **Steps:**
  1. Seal `DATABASE_URL`, commit the SealedSecret, add `envFrom` to the Deployment.
  2. Search the repo for the plaintext value.
  3. Back up the sealing key outside Git.
- **Done when:** the Secret exists in `shop-dev`, the plain value is absent from the repo and history, and the key backup exists.

☐ **T8 — CI hand-off** *(≈75 min, advanced; Milestone 6; topic 7)*

- **Goal:** a push to the app repo updates the dev overlay.
- **Steps:**
  1. Create a fine-grained token limited to `shop-gitops` contents.
  2. Add the CI workflow to the app repo and push a change.
- **Done when:** a `ci-bot` commit appears in `shop-gitops` and Argo CD deploys the new SHA tag to dev.

☐ **T9 — Roll back with git revert** *(≈60 min, advanced; Milestone 7; topic 6)*

- **Goal:** recover from a bad prod promotion.
- **Steps:**
  1. Promote a broken tag to prod and watch it degrade.
  2. Recover with `git revert`, then repeat using the emergency path and re-enable automation.
- **Done when:** prod is `Synced` and `Healthy`, history shows promotion plus revert, and auto-sync is back on.

☐ **T10 — Prove it: rebuild from scratch** *(≈90 min, advanced; all milestones; all topics)*

- **Goal:** show the repo really is the source of truth.
- **Steps:**
  1. `kind delete cluster --name gitops`, create a new cluster, reinstall Argo CD, re-apply only `root-app.yaml` (and restore the sealing key).

2. Write a one-page explanation of what was restored automatically and what was not (secrets key, Argo CD admin password).

- **Done when:** all three environments return to Healthy with the same image tags as before, and your write-up lists the remaining manual steps.

6 Verification checklist

- [ ] `kubectl -n argocd get applications` shows `root` and three environment Applications, all `Synced` and `Healthy`.
- [ ] A manual `kubectl scale` on dev is reverted without human action.
- [ ] Deleting a manifest from Git deletes the live object (prune) and a `Prune=false` object survives.
- [ ] A bad PreSync hook blocks a release; a good one runs before the rollout and `smoke-test` after.
- [ ] Each promotion is a PR with a one-line diff; prod changes need a code-owner approval.
- [ ] No plain-text secret exists in the repo or its history (`git log -p | grep CHANGE_ME` returns nothing).
- [ ] CI pushes an image and a bot commit bumps the dev tag; no cluster credentials exist in CI.
- [ ] A `git revert` restores a previous prod version, and the audit trail shows who did what.
- [ ] The whole stack rebuilds from `root-app.yaml` on a new cluster.

7 Teardown

```bash
kill %1 2>/dev/null                    # stop the port-forward if it is still running
kind delete cluster --name gitops
```

Delete the `shop-gitops` / `shop-api` GitHub repositories and the fine-grained token if you no longer need them. Remove the Actions secret `GITOPS_TOKEN`. Keep the sealing-key backup only as long as you need it; it is a credential.

8 Stretch goals and what to learn next

- Replace the hand-built Applications with Flux on a second cluster and compare (Argo CD and Flux §17.2).
- Add Argo CD Image Updater or Flux image automation for dev, keeping staging and prod PR-gated.
- Use the ApplicationSet `pullRequest` generator to create a preview environment per app PR.
- Add Argo Rollouts for canary releases driven from Git (Canary Deployments §5.12).
- Restrict teams with `AppProject`s and Kubernetes RBAC (RBAC in Depth §16.1).
- Swap Sealed Secrets for External Secrets Operator backed by AWS Secrets Manager.
- Next level: Level 18 - Platform Engineering §18.1, where GitOps becomes the delivery engine for a self-service internal platform.